Iteration 4 — Revision Report

Where the 6.2% yaw discrepancy really lives, and what that withdraws from the original report

What this document is

change_report_4.pdf stands as issued. Nothing in it has been edited and none of its figures or measurements have been recomputed. This is a separate record of what the follow-up analysis changed, written because one of its central recommendations rested on a mechanism that turns out not to exist.

The two file changes made in iteration 4 — the recovery round robin in `recovery_nav_bt.xml` and `velocity_smoother.smoothing_frequency` in `Course2027.yaml` — are **unaffected** by everything below. No further file changes were made. Every parameter named in this report lives outside the two files I am permitted to modify.

The one-line version

The 6.2% is real, but it is a **command-path gain error**, not an estimation error. It cannot reach your heading estimate, so it cannot be the cause of the cost map smear that section 7 item 1 of the original report blamed it for. The gyro is the honest sensor and the wheels are the liars — and now that the track is confirmed to 1.3 mm, `wheel_radius` is the leading suspect, not `wheel_separation`.

1. Withdrawn: `wheel_separation` cannot corrupt the heading estimate

Section 7 item 1 of the original report called the 6.2% “the largest recoverable clearance source” and explained it through cost map smear: a yaw-scale error rotating obstacles as they are marked. **That mechanism is withdrawn.** It requires the wheel-derived yaw to reach the transform that STVL marks voxels with, and it does not.

The local EKF never fuses a wheel-derived yaw

Decoding the `Local_EKF.yaml` configuration vectors, whose order is `[x y z roll pitch yaw vx vy vz vroll vpitch vyaw ax ay az]`:

source	indices set true	what is actually fused
`odom0` — `/diffbot_base_controller/odom`	6, 7	`vx` and `vy` only. `vyaw` (index 11) is **false**, so the wheel-reported yaw rate is discarded. Fusing a hard `vy` of 0 acts as a nonholonomic constraint, which is fine.
`imu0` — `imu/data`	11	`vyaw` only — one scalar, `angular_velocity.z`. Absolute `yaw` (index 5) is **false**, and so are both acceleration channels, which makes `imu0_remove_gravitational_acceleration: true` a no-op.

So the fused yaw rate is the gyro, alone. And the wheel contribution is linear velocity, where

```
v = r · (ω_R + ω_L) / 2
```

has no *b* in it at all. The track error is algebraically absent from every quantity the EKF consumes.

And it is the EKF that drives the frames

Measured in the bags: `odom` → `base_link` arrives at exactly **30.0 Hz**, matching `ekf_local.frequency: 30.0`, from a single publisher. Every cost map in `Course2027.yaml` has `global_frame: odom`, and `controller_server.odom_topic` is `odometry/filtered/local`. So both the marking transform and the controller's feedback come from the EKF, whose yaw is the gyro.

> **Loose end worth a look**
>
> `diffbot_controllers.yaml` sets `enable_odom_tf: true`, and `robot_localization` publishes the same transform by default. Two publishers of `odom` → `base_link` would be a real problem. Empirically there is only one, at the EKF's rate, so the controller's transform is not reaching `/tf` — but nothing in the configuration explains why, and it would be better to set that flag to false deliberately than to rely on whatever is currently suppressing it.

2. What the 6.2% actually is: a command-path gain error

`diff_drive_controller` inverts the kinematics using its configured track to turn a commanded yaw rate into a wheel-speed difference:

```
Δv = ω_commanded · b_config
ω_actual = Δv / b_true = ω_commanded · (b_config / b_true)
```

With `b_config` 0.856 and an effective track of 0.806, that ratio is 1.062: **every turn is 6.2% sharper than asked for.** This is a plant gain error on the yaw channel, and no EKF setting participates in that arithmetic.

It also means the 1.062 diagnostic is measured between `/imu/data` and `/diffbot_base_controller/odom` — two *raw* topics. The EKF is downstream of both and feeds back into neither, so that ratio will read 1.062 regardless of what is fused.

symptom	severity	why
Steady-state path error	**None measurable.**	MPPI closes the loop on the EKF estimate, whose yaw rate is the gyro — the truth. A constant 6.2% gain error is trivial for a feedback loop to absorb. The original report's own measurement confirms it: cross-track error 3–10 mm median through the corner.
Transient overshoot at curvature changes	**Real, and this is the cost.**	The yaw loop carries about 300 ms of command-to-measurement lag (~45 ms transport, ~190 ms motor time constant, 47 ms EKF). Inside that window the controller is effectively open-loop and the plant delivers 6.2% more rotation than any model predicted. MPPI's rollouts assume the commanded ω is achieved, so each 100 ms cycle optimises against a slightly wrong plant and re-corrects. Expect mild under-damping entering and leaving turns, worst where curvature changes fastest — which is the 1.25 m pinch.
Wheel odometry's own pose and `twist.angular.z`	Wrong by 6.2%, and inert.	Nothing subscribes to them for yaw. A wrong number with no consumer, which is exactly why it survived three iterations unnoticed.
Direction of the error	Unfavourable.	1.062 > 1 means the robot out-rotates its command, so with that lag it overshoots *into* turns. The robot cuts corners tighter than planned — the wrong way for a clearance goal.

3. The gyro is right; the wheels are wrong

A gyro scale error of (k–1) corrupts integrated heading by (k–1) multiplied by the *net* heading change. `scan_matcher` observes absolute heading, so whatever correction it had to apply to `map` → `odom` over a run

bounds the gyro's accumulated error. Integrating both rate sources over each bag and comparing:

run	net heading turned	gyro / wheel ratio	scan-matcher yaw correction	implied gyro scale error
real_bag1	+168.4°	1.0394	+0.77°	0.5%
real_bag2	–63.7°	1.0815	+1.23°	1.9%
real_bag3	+84.6°	1.0345	+0.08°	0.1%
real_bag4	+104.8°	1.0515	–0.09°	0.1%
real_bag5	+107.8°	1.0585	+0.44°	0.4%
real_bag6	+96.3°	1.0625	+0.07°	0.1%
real_bag7	–9.1°	*0.8997*	–1.01°	*meaningless — only 9° of net turning, so the ratio is noise*
real_bag8	–165.6°	1.0662	–0.75°	0.5%
real_bag9	+46.4°	1.0718	–0.67°	1.4%
real_bag10	–71.7°	1.0648	–0.47°	0.7%

If the gyro carried the 6.2%, run 1 would have needed a **10.4°** correction. It needed 0.77°. The gyro is good to well under 1% in every run with meaningful rotation, so the wheel-reported yaw rate is the wrong one.

Caveat: `scan_matcher` has its own error, so these corrections bound the gyro's error rather than measuring it exactly. The bound is tight enough to settle a question about 6.2%.

4. Revised suspect: `wheel_radius`, not `wheel_separation`

The centre-to-centre track has now been measured precisely as **0.85486875 m** (33 + 10.5/16 inches) — **1.3 mm** from the configured 0.856. Centre-to-centre is also the kinematically correct quantity: the no-slip constraint acts at the contact patch, which for a thin wheel sits in the wheel's mid-plane. So the geometric track is right, and the 48.6 mm of effective-track error is not a mis-measurement.

Since $\omega_wheel = r(\omega_R - \omega_L)/b$ and b is now confirmed, the remaining single-parameter explanation is r:

```
r_true = 0.165 × 1.062 = 0.1752 m
```

a 13.8" loaded diameter against the configured 13", which is entirely ordinary for a pneumatic tire. Three things favour this over a track error:

argument	detail
The direction works	Contact-patch scrub across a 102 mm tire and two trailing swivel casters ~0.79 m behind the axle both make the *effective* track **larger** than geometric, never 49 mm smaller. A track error of this sign has no physical mechanism behind it; a radius error needs none.
One parameter explains both channels	r scales linear and angular odometry by the same factor. A track error explains only the angular one, leaving the linear channel to be separately correct by coincidence.
Weak corroboration	`linear_calibration.py` reads 1.021 and 1.027 on bags 1 and 2 — the same sign. But r^2 is 0.07 and 0.02, so those fits are **unusable as evidence**; they are mentioned only because they do not contradict.

The test that settles it, and it takes ten minutes

Tape 10 m on the floor, drive it straight, read the odometry displacement. *b* cannot appear in linear odometry, so this isolates *r* completely.

• Reports about **9.42 m** for 10 m driven → *r* is the whole problem and the track never needed touching.

• Reports **10.0 m** → *r* is right, the effective track genuinely is 0.806 m, and that is a real physical puzzle worth chasing.

Do this **before** changing any parameter. Then, and only then, get *b* from a figure-of-eight (equal left and right arcs, so gyro bias cancels) or a UMBmark square driven five times each way — not an in-place spin, which maximises scrub and caster drag and so measures an effective track that applies only to spins.

5. Conditional, and time-sensitive: a speed-compliance risk

If the radius hypothesis is right, the robot is over the IGVC speed limit

A `wheel_radius` that is 6.2% too small means true speed is 1.062 × reported. `max_vel_x: 2.235` m/s, chosen as exactly 5 mph, would really be **2.374 m/s = 5.31 mph**.

Even the 2–3% from those unusable linear fits would put it at 5.1–5.15 mph.

This is conditional on the hypothesis, and the straight-line test resolves it. Worth resolving before the next scored run.

6. Do not switch on `imu0_config` index 5

Enabling absolute yaw was considered as a way to correct the discrepancy without touching the track. It would not: section 2 shows the error is on the command path, which no EKF setting reaches. But the more important finding is that the UM7's absolute yaw is not fit to fuse regardless. Comparing the orientation quaternion's yaw against a pure integral of the same message's `angular_velocity.z`:

run	orientation yaw, net	gyro integral, net	residual
real_bag1	+155.81°	+168.44°	–12.63°
real_bag3	+118.36°	+84.61°	**+33.76°**
real_bag8	–142.06°	–165.62°	+23.56°
real_bag10	–50.87°	–71.71°	+20.84°

They diverge by 12–34° per run, so the orientation is magnetometer-referenced and wrong by that much. The residual is not a consistent scale factor — bag 3 over-reads while the others under-read — which is the signature of magnetic distortion rather than anything systematic. Steel structure and motor currents in a lab and a hallway will do exactly this.

The dangerous part is the declared covariance

`orientation_covariance[8]` is **0.00274 rad²**, i.e. σ = **3.0°**. The UM7 claims 3° confidence while being off by 34°. Fusing that would drag the heading toward a badly wrong value *with high confidence* — substantially worse than the 2.5–4.3° of drift currently present.

If an absolute heading reference is wanted, take it from `scan_matcher`, not the magnetometer.

7. Replacement explanation for the cost map smear

The original report measured obstacle thickness at 5–14 cm where a flat wall should be one or two 2.5 cm cells, and attributed it to the yaw-scale error. That attribution is withdrawn with section 1. The replacement explanation is structural, and larger:

There is no absolute heading reference in the frame the cost maps live in

`imu0_config` index 5 (yaw) is **false in both EKFs**, so heading is an unbounded integral of a 19.6 Hz gyro rate.

Every cost map in `Course2027.yaml` — local, global, and the behavior server's — has `global_frame: odom`.

`scan_matcher` does produce absolute corrections, but they land in `map` → `odom`, and nothing in the navigation stack reads the map frame. The corrections are computed and then ignored.

Measuring the correction `scan_matcher` applies gives the disagreement the cost maps never see. Final values stay inside ±1.2°, so there is no large monotonic drift — but the correction **swings over 2.5–4.3° within every single run**, in steps:

run	final correction	range within run
real_bag1	+0.77°	**4.32°**
real_bag2	+1.23°	2.50°
real_bag3	+0.08°	3.76°
real_bag4	–0.09°	2.58°
real_bag5	+0.44°	2.55°
real_bag7	–1.01°	2.70°
real_bag9	–0.67°	**4.23°**
real_bag10	–0.47°	**4.07°**

At 3 m range, 3° is 0.157 m of tangential error — the right order for the 5–14 cm thickness measured, and well within what an unreferenced gyro accumulates over the 4 s STVL holds a voxel. Same caveat as section 3: the scan matcher has its own error, so this measures *disagreement* and is an upper bound on gyro drift rather than a clean reading of it.

If this holds up, fusing absolute yaw from `scan_matcher` would buy more clearance than fixing the wheels will. It is also the one change that attacks the smear directly rather than through the controller.

8. Two geometry discrepancies found in passing

Both outside the two permitted files, so recorded and not changed.

The URDF disagrees with the tape

`diffbot_description.urdf.xacro` places the wheel centres at `base_width/2 + wheel_yoff` = 0.3112 + 0.124 = 0.4352 m per side, a centre-to-centre of **0.8703 m**. The measured value is **0.85487 m** — **15.4 mm** apart, 7.7 mm per side. Since the footprint's half width is derived from these same properties, the error propagates into the cost map.

The footprint is 11.6 mm per side conservative

Outer tire face from the measured track is 0.85487/2 + 0.102/2 = **0.47843 m**; the configured footprint uses **0.49 m**. Correcting it recovers support width across a corridor:

heading relative to the corridor	configured footprint	corrected	recovered
0° (square-on)	0.9800 m	0.9569 m	**23.1 mm**
4° (start → finish)	1.0069 m	0.9838 m	23.1 mm
20°	1.1399 m	1.1291 m	10.9 mm
26° (finish → start)	1.2105 m	1.2001 m	10.4 mm
30°	1.2501 m	1.2401 m	10.0 mm

Through the 1.25 m pinch the hard attitude limit moves from **30.0° to 31.1°**. That is a modest gain and it does not change the original report's conclusion about that corner — but the whole argument there is about centimetres, and this is one of them, available for the price of a corrected constant. Note the direction: the current footprint is *conservative*, so this is recovering real margin, not discovering a safety gap.

9. Also corrected: the local cost map was never missing its update rate

Separate from the yaw work, and a smaller correction. The iteration-1 comments in `Course2027.yaml` attribute the low `/costmap` rates to load. Part of it was never load.

`Costmap2DROS::updateMap()` publishes the oriented footprint unconditionally, once per update-loop iteration, and it is *not* gated by `publish_cycle_`. So `published_footprint` is the update loop's heartbeat:

topic	configured	measured	reading
`local_costmap/published_footprint`	update 10.0 Hz	9.96–10.02 Hz; median **100.0 ms**, p90 100.2–101.3, max 106.1 ms, no cycle doubled in any of the ten bags	The update loop hits its rate exactly.
`local_costmap/costmap`	publish 10.0 Hz	6.35–6.60 Hz; intervals are 100 ms 46–49% of the time and 200 ms 51–56%, nothing else	Aliasing, not a missed deadline.
`global_costmap/published_footprint`	update 5.0 Hz	4.97–5.01 Hz; median 200.0 ms	Also exact.
`global_costmap/costmap`	publish 5.0 Hz	3.13–3.36 Hz; intervals 200 or 400 ms	Same aliasing, one octave down.

The publish gate is a strict inequality:

```
if ((last_publish_ + publish_cycle_ < current_time) || ...)
```

With `publish_frequency == update_frequency`, `publish_cycle_` is 100 ms and the loop ticks every 100 ms, so each cycle the comparison decides whether ~100.0 ms is strictly greater than 100.0 ms — a coin flip on microseconds of jitter. Both cost maps have publish equal to update, which is why both alias.

consequence	detail
nav2 cannot warn about this	The *"Map update loop missed its desired rate"* warning sits inside an `#if 0` block with a `TODO(bpwilcox)` — `r.cycletime()` was never ported to ROS 2. A quiet `/rosout` is not evidence; the footprint topic is the only observable.
`costmap_raw` rides the same gate	`Costmap2DPublisher::publishCostmap` sends `/costmap` and `/costmap_raw` together, so the behavior server — which subscribes to `local_costmap/costmap_raw` for its `isCollisionFree` check — re-checks at `cycle_frequency` 10 Hz against a map that changes at 6.4 Hz. Not a safety issue, but about a third of those checks are duplicates.
Nothing a planner or controller uses is affected	`controller_server` owns the local cost map object in-process and never reads the topic. The only real cost is to bag analysis, which has been interpolating between 6.4 Hz frames.
Optional fix	`publish_frequency: 20.0` makes the gate always already due. Nearly free when nobody is subscribed, since both publishes are guarded by `get_subscription_count() > 0`. When bagging it is a 160 kB `OccupancyGrid` ten times a second instead of 6.4. This is inside `Course2027.yaml` and was **not** changed, since it affects diagnostics only.

10. What still stands from the original report

Unaffected by any of the above, and restated here only so the revision is not mistaken for a broader retraction:

finding	status
The 1.25 m pinch, and the attitude arithmetic that makes finish→start harder than start→finish	**Stands.** Section 8 shifts the hard limit by 1.1°; the argument is unchanged.
Both the planner's `cost_penalty` and MPPI's `CostCritic` scoring the centre cell while the footprint is only a binary veto	**Stands.** Independently verified in nav2's source and by replay.
Every BackUp aborting with error 714, and the round-robin reordering that addresses it	**Stands.** Both file changes are unaffected.
No parameter configuration fixes the clearance (smoother off, `cost_penalty` 3/8/15, `cost_scaling_factor` 1.7/1.3/1.0, two lattices; footprint minimum pinned at 0.025 m throughout)	**Stands.** Replay result, independent of the yaw question.
The IMU latency verdict — `latency_timer` fix live and worth 42.6 → 24.8 ms, rate change never built	**Stands.** Section 3 adds that the gyro's *scale* is also good, which was not previously established.
Iteration 3's correction: ~45 ms dead time, not 280–320 ms	**Stands.**

11. Revised recommendations, in order

#	action	where	why now
1	Drive a taped 10 m straight line and compare to odometry displacement	no file change — a measurement	Isolates *r* from *b* completely. Every other number here depends on the answer, and it costs ten minutes. **Do this first.**

#	action	where	why now
2	Re-check speed compliance once *r* is known	`Course2027.yaml` `max_vel_x`	Possibly 5.31 mph against a 5 mph limit. The only item here with a rules consequence.
3	Then measure *b* by figure-of-eight or UMBmark, and apply the result as `wheel_separation_multiplier` / `*_wheel_radius_multiplier` rather than by editing the measured constants	`diffbot_controllers.yaml`	Keeps 0.85487 as the honest physical measurement with the empirical correction visible beside it. The multipliers are currently 1.0 and exist for exactly this.
4	Consider fusing absolute yaw from `scan_matcher`; do **not** enable `imu0_config` index 5	`Local_EKF.yaml`	Attacks the smear directly. Section 7 argues this is worth more clearance than the wheel fix; section 6 is why the magnetometer is not the way to get it.
5	Correct `wheel_yoff` (or `base_width`) to match the tape, and the footprint's half width with it	`diffbot_description.urdf.xacro`, `Course2027.yaml`	23 mm of real margin square-on, 10 mm at the obliqueness that matters. Small, but this corner is decided in centimetres.
6	Set `enable_odom_tf: false` explicitly	`diffbot_controllers.yaml`	Currently relying on something unidentified to suppress a duplicate transform publisher.
7	Build the unbuilt `update_rate: 60` launch edit	`colcon build --packages-select imprimis_hardware_platform`	Carried over from the original report, still outstanding. Accounts for the entire 24.8 ms of residual IMU lag.

12. Tools added for this analysis

All in `claude_tools/`, per the standing convention.

tool	what it does
`gyroscale.py`	Integrates gyro and wheel yaw rates per bag and compares the net heading change against the scan matcher's applied correction. This is the tool that settles gyro versus wheels.
`imuyaw.py`	Compares `/imu/data`'s orientation yaw against an integral of its own gyro rate, to establish whether the absolute yaw is magnetometer-referenced and whether it is usable.
`yawdrift.py`	Extracts the `map` → `odom` yaw correction over a run, as a proxy for the heading drift the cost maps are exposed to and never see corrected.
`loop_rate.py`	Separates cost map update rate from publish rate using `published_footprint` as the loop heartbeat, with an interval histogram that exposes the aliasing.
`tfdup.py`	Counts publishers and rates per transform pair, to detect duplicate `odom` → `base_link` publication.
`support.py`	Computes the footprint's support width across a corridor as a function of obliqueness, for the configured and tape-corrected half widths.
`make_revision_report_it4.py`	Builds this document.

13. What to measure on the next run

#	measurement	settles
1	Taped 10 m straight line versus odometry displacement	*r* versus *b* — the open question this whole report rests on.
2	Figure-of-eight with equal arcs, gyro versus wheel integrated yaw	*b*, once *r* is fixed, without the scrub that an in-place spin adds.

#	measurement	settles
3	Record `local_costmap/costmap_raw` and `global_costmap/costmap_raw`, and fix the `/speed_governor_info` typo in the bag command	Third iteration carrying this request. Would let clearance be measured on the map the behavior server actually reads.
4	A stationary 60 s hold with the gyro logged	Gyro bias stability, which sets how fast the unreferenced heading in section 7 drifts.
5	`scan_matcher` yaw alongside EKF yaw throughout	Whether a scan-matcher yaw fusion would have the quality to be worth adding.

Prepared as a revision to `iterations/4/change_report_4.pdf`, which is unmodified. No source files were changed in producing this document; the two iteration-4 file changes stand as reported.